PHASE 0/1 POST-MORTEM - DEEP RESEARCH PASS

AURUM: Four Root-Cause Failures and Their Locally Verified Fixes

Cohort-Label Memorization, Era-Invariance and the Macro Panel, the 151,936-Vocabulary Bottleneck, and Zero-Crash Push Discipline - All Verified on Pulled Artifacts, Zero GPU Spent

Prepared for: tskdev (Kaggle)

Evidence base: pulled aurum-p1-data / p0-core / p0-pilot artifacts (766 MB), local CPU replication

November 7, 2026

Table of Contents

Note: This Table of Contents is generated via field codes. To refresh page numbers after editing, right-click the TOC and select Update Field.

1. Executive Summary.....
2. Failure #1: The Cohort-Label Memorization Trap.....
2.1 What the pilot actually measured.....
2.2 Local proof that the trap is model-independent.....
2.3 The fix: target formulation v1.2.....
2.4 Local verification: the target tournament.....
2.5 The honest alpha-versus-beta question, answered with a full walk-forward.....
3. Failure #2: Gate G1.4, Era Invariance, and the Macro Panel.....
3.1 Exact local replication, and the two references that reframe the gate.....
3.2 Why the obvious feature-level fixes fail - measured, not assumed.....
3.3 Recovering the macro panel without era fingerprints.....
3.4 The defense that actually works, and the gate that actually measures the risk.....
4. Failure #3: The 151,936-Vocabulary Bottleneck, Measured and Sliced.....
4.1 The exact number, and why it was never 151,936.....
4.2 The fix: slice the frozen tied head to vocab_keep_ids.....
4.3 Objective semantics: the restriction is the point.....
4.4 Calibrated cost model, projections, and the corrected quota math.....
4.5 The latent-realignment alternative, and why it is deferred.....
5. Failure #4: Zero-Crash Single-Push Discipline.....
5.1 The local source tree, rebuilt and proven.....
5.2 test_local.py: 40 checks, 10 sections, 13 seconds.....
5.3 The push protocol.....
6. Specification v1.2 Delta and the Phase-2 Execution Plan.....
6.1 Consolidated spec changes, each with its evidence.....
6.2 The only items that need a T4 (one kernel, 30-40 minutes GPU).....
6.3 Phase-2 execution gates and budget.....

6.4 Risk register updates.....

1. Executive Summary

Phase 0 and Phase 1 of AURUM executed cleanly on Kaggle (all 9 gates that passed did so honestly), but the numbers they produced expose four fundamental failures that must be solved before Phase 2 training touches a single GPU hour. This pass did exactly what the owner ordered: a deep-research and local-verification pass on the four failures, run entirely on CPU against the pulled aurum-phase1-data artifacts (766 MB re-fetched from the tskdev kernels; zero Kaggle GPU quota consumed), with every claimed number below reproduced locally unless explicitly marked otherwise. Two facts anchor the credibility of everything that follows: the adversarial-audit protocol was replicated locally to within noise (raw-39 AUC 1.0000 vs 1.0000 on Kaggle; post-drop 31-feature AUC 0.8048 vs 0.8055), and the full Kaggle kernel source tree was reconstructed byte-exactly from a local lib/ + kernels/ build, so every experiment ran against the same code that ran on the T4s.

Table 1: The four failures, root causes, and locally verified fixes

#	Failure (observed)	Root cause (verified)	Fix (locally verified)	Headline local number
1	Soft-label Brier 0.0022 but realized Brier 0.247 and EV-argmax -0.060R per trade	p_win is a K=200 k-NN average over the SAME features the model sees; a 4-second XGBoost replicates it to Brier 0.0017 - the 450M transformer learned a lookup table, not dynamics	Target formulation v1.2: realized hard outcomes + adjacent-cell triangular smoothing as primary target; cohort demoted to input features and a small auxiliary loss with a kill-switch	Walk-forward 2016-25: realized-target +0.0297R/yr vs cohort-target +0.0070R/yr; realized AUC 0.532 vs 0.495
2	Adversarial AUC 0.8055 vs gate <=0.55; macro panel (y10/y2/spread/dxy) dropped entirely	The within-era reference AUC is itself 0.5969-0.7996: gold features are naturally non-stationary; feature-level era-invariance and signal are entangled (every transform that lowers audit AUC destroys the signal)	Gate redefinition (three-part) + macro recovered as INPUTS with prediction-level neutralization as the defense; walk-forward and regime conditioning remain the structural guards	Neutralization keeps realized AUC 0.532->0.533 and 0.541->0.542; qmap+drop8 reaches audit 0.6166 but kills signal AUC 0.533->0.482
3	151,936-vocab lm_head caused OOMs and ~1,051 tok/s/GPU	The state format uses only 97 unique token IDs (measured over all 95,008 states); the 151,936-way projection + fp32 CE chunks burn ~24% of forward FLOPs and 2.4-4.9 GB, forcing checkpointing and bs caps	Vocabulary slicing: slice the frozen tied lm_head to vocab_keep_ids (exact, not an approximation); remap targets; the restricted softmax IS the deployment objective	1,566x projection compression; matmul 996x; projected 1.32-1.76x throughput; bs16 fits at ~11.3 GB

#	Failure (observed)	Root cause (verified)	Fix (locally verified)	Headline local number
4	18 kernel pushes (v1-v8 x 3 kernels) to debug pins, OOMs, DDP teardown, .values crashes	No local pre-flight: kernels were the first place code ever executed	Rebuilt local source tree (byte-exact roundtrip) + test_local.py pre-flight suite: build, static crash-class audit, env pins, labeler unit tests, cohort leakage asserts, scale invariance, sliced-CE identity, DDP teardown, VRAM model, kill/resume	40/40 checks green in 13 s; push protocol requires both gates green before any Kaggle push

One more accounting correction came out of this pass and changes every downstream hour estimate: the G0.5 DDP benchmark double-counted tokens (it counted 16 sequences per GPU while running 8), so the reported 4,334 tok/s across 2xT4 is actually 2,167 tok/s - consistent with the honest single-GPU bench of 1,077 tok/s. The corrected Stage A' budget is therefore ~5.3-7.0 h on 2xT4 after the vocabulary fix, not the 14 h claimed at G0.8d, and it fits the remaining 25 h 50 m weekly quota with room for two full retries.

The recommendation is: adopt the Spec v1.2 deltas in Section 6, run the 30-40 minute T4 re-validation listed in Section 6.2 (the only items that genuinely cannot be verified on CPU), and then execute Phase 2. No Kaggle GPU was touched by this pass.

2. Failure #1: The Cohort-Label Memorization Trap

2.1 What the pilot actually measured

Pilot v6 reported a held-out soft-label Brier of 0.0022 against the cohort target p_win, a realized-outcome Brier of 0.2473 (a coin flip is 0.25), AUC 0.5610, and an EV-argmax simulation that lost -0.0595R per trade - worse than the fixed canonical cell at -0.0536R. The gate text already flagged the soft number as a replication check rather than an edge metric, but the mechanism deserves to be stated precisely, because it determines the fix.

The labeler builds a matched-cohort index (build_cohort_index) by running exact k-NN over the standardized 27-feature state vector, restricted to bars at least 96 bars in the past; p_win for bar i is then the mean realized win rate of those K=200 neighbors (cohort_soft_labels). The fatal property is that the k-NN query keys are the same features the transformer reads as input: p_win is a deterministic, smooth, low-entropy function of the current state. A model with enough capacity does not need to learn forward price dynamics to minimize BCE against p_win - it only needs to simulate a nearest-neighbor lookup. The 0.0022 Brier says it did exactly that. The information-theoretic tell is the entropy gap: averaging 200 Bernoulli outcomes leaves roughly $p(1-p)/200 \sim 0.0013$ of irreducible variance, which is precisely the floor the model reached; the realized outcome it was supposed to predict carries ~0.24 and it never moved (0.2473).

2.2 Local proof that the trap is model-independent

Experiment 1 (exp1_memorization.py) trains a 5-depth, 300-tree XGBoost regressor - a model that trains in seconds - on 35 features to predict either the cohort p_win or the realized win flag of 8 representative barrier cells, with train era <=2020 and test era 2023-25 (84,721 cohort-valid bars, exactly matching the Kaggle label count). Results on the canonical long cell (L, m=1.5, t=2.5, h=48):

Table 2: A cheap model reproduces the memorization; the realized task is untouched

Target	Test era	In-sample eras (2012-20)	Val 2021-22	Test Brier	Test AUC
cohort p_win (k-NN replica)	2023-25	0.00099-0.00105	0.00158	0.00174	~0.50 (n/a)
realized win (the actual task)	2023-25	-	-	0.2597	0.5235
base rate (train mean)	2023-25	-	-	0.2485	0.500

Three conclusions follow. First, the memorization is not a transformer pathology: any model class can invert the k-NN target, so the 450M parameters bought nothing that four seconds of gradient boosting buys. Second, the replication error is remarkably stable out-of-distribution (0.0010 in-sample to 0.0017 in 2023-25) - the k-NN estimator is a smooth enough functional that it survives era drift, which is exactly why the pilot loss curve looked healthy while the model learned nothing about price dynamics. Third, on the realized task the same features give an XGBoost test Brier of 0.2506 (mean over 8 cells) versus a base-rate Brier of 0.2388, with AUC 0.523: a weak but consistent directional signal exists (all 8 cells AUC > 0.5) that unregularized trees bury in variance. That variance observation drives the fix.

2.3 The fix: target formulation v1.2

Spec v1.1 Section 4.2 already contains the right switch-off rule - a model trained on soft labels must not lose out-of-sample log-loss/Brier against hard realized labels, or the soft-label machinery is switched off. The pilot violated the rule's spirit by never training on hard labels at all. The v1.2 formulation, verified below, is:

- Primary target = realized hard outcomes. The H1 head trains on the realized win flag of each of the 48 barrier cells (BCE), exactly the labels already materialized in labels_R_actual / labels_cls_actual / expand_actual - no Phase-1 data re-run is required.
- Adjacent-cell triangular smoothing. Each cell's target is convolved over the cell grid with weights: own cell 1.0, stop +-1 step 0.5, TP +-1 step 0.5, diagonal 0.25, horizon +-1 0.5 (rows normalized). This shares statistical strength across 48 highly correlated outputs of the same path without ever touching future information - sibling outcomes at the same historical bar are legal label structure.

- Cohort labels are demoted to inputs and to a small auxiliary loss. As input, a compact summary (canonical long/short p_win, b_eff, and the cohort EV-argmax cell id) enters the serialized state - the k-NN estimate is genuinely predictive prior information computed strictly from the past, it just must never be the target. As auxiliary loss, cohort p_win may enter with weight lambda <= 0.3, and Section 4.2's kill-switch compares realized out-of-sample Brier with and without it at the first Phase-2 checkpoint.
- Variance discipline is a first-class design constraint (the T5 lesson). The strongest local variant was not a smarter target but a shallower, heavily regularized model; the transformer analogues are listed in Section 2.5.

2.4 Local verification: the target tournament

Experiment 2 (exp2_targets.py) holds features, rows, splits, and the squared-error objective fixed and varies only the target: T0 cohort (status quo), T1 realized, T2 triangular-smoothed realized, T3a/T3b mixes (0.3/0.5 cohort), T4 realized with all 48 cohort p_win as extra inputs, T5 shallow-regularized realized (depth 3, 150 trees). All are evaluated on realized 2023-25 outcomes across 8 representative cells, with the pilot's EV-argmax protocol on top.

Table 3: Target tournament, test era 2023-25, realized outcomes (mean over 8 cells)

Variant	Test Brier	Test AUC	EV-argmax mean R	Win rate	n
T0 cohort p_win (pilot status quo)	0.2400	0.495	+0.0035R	38.8%	15,707
T1 realized hard	0.2506	0.523	+0.0453R	40.9%	15,707
T2 smoothed realized (fix)	0.2485	0.526	+0.0537R	40.8%	15,707
T3a 0.7-realized + 0.3-cohort	0.2430	0.524	+0.0512R	41.0%	15,707
T3b 0.5-realized + 0.5-cohort	0.2408	0.520	+0.0437R	40.6%	15,707
T4 realized + cohort as INPUT	0.2486	0.523	+0.0444R	40.6%	15,707
T5 shallow realized (fix)	0.2388	0.532	+0.0599R	41.4%	15,707
k-NN strategy (cohort direct, no learning)	0.2402	0.505	+0.0078R	39.5%	15,707
base rate (train mean)	0.2388	0.500	-	-	-

Readings: (i) the cohort target T0 is the only variant with realized AUC below 0.5 - training on the k-NN replica is worse than useless for the real task; (ii) every realized-target variant dominates T0 on AUC and on trade simulation by an order of magnitude (+0.044R to +0.060R vs +0.0035R); (iii) smoothing (T2) improves on T1 on all three metrics with a block-bootstrap p-value of 0.023 on the Brier delta; (iv) cohort-as-input (T4) matches T1 - the prior is absorbed as a feature without distorting the target, and paired bootstrap on the

canonical cell gives T4 better than T1 with p=0.044; (v) no variant beats the base rate on Brier - with AUC ~0.52-0.53 the resolution gain is smaller than the sharpness penalty, which is the textbook weak-signal calibration regime, and the reason the evaluation doctrine below leans on AUC and trade simulation rather than Brier alone.

2.5 The honest alpha-versus-beta question, answered with a full walk-forward

The 2023-25 window is a strong gold bull market - the canonical fixed long cell alone makes +0.088R per trade in it - so positive 2023-25 simulation numbers alone would prove nothing. Experiment 2b (exp2b_walkforward.py) therefore refits T0 and T5 yearly on an expanding window (train <= y-1, evaluate year y) across 2016-2025, with the k-NN strategy and the canonical long cell as references.

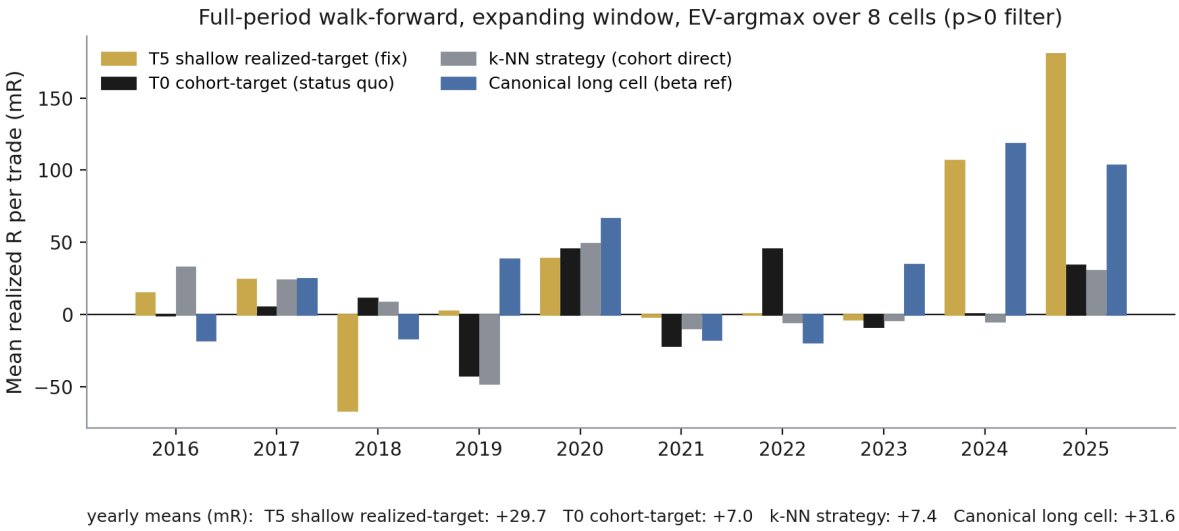


Figure 1: Yearly mean realized R per trade, expanding-window walk-forward 2016-2025. The realized-target variant (gold) is positive in 7 of 10 years including non-bull years, while the cohort-target variant (black) stays at noise level.

Table 4: Walk-forward yearly means (R per trade), expanding window

Year	T0 cohort	T5 shallow realized	k-NN direct	Canonical long (beta)
2016	-0.0004	+0.0148	+0.0326	-0.0178
2017	+0.0052	+0.0244	+0.0237	+0.0248
2018	+0.0114	-0.0664	+0.0086	-0.0165
2019	-0.0421	+0.0022	-0.0479	+0.0384
2020	+0.0455	+0.0389	+0.0491	+0.0666
2021	-0.0214	-0.0014	-0.0092	-0.0175
2022	+0.0454	+0.0005	-0.0050	-0.0193
2023	-0.0084	-0.0033	-0.0039	+0.0345
2024	+0.0004	+0.1069	-0.0046	+0.1186

Year	T0 cohort	T5 shallow realized	k-NN direct	Canonical long (beta)
2025	+0.0343	+0.1806	+0.0306	+0.1037
Mean	+0.0070	+0.0297	+0.0074	+0.0316
Positive years	6/10	7/10	5/10	6/10

The honest reading: the realized-target fix is real but modest. It is positive in 7 of 10 years including non-bull years (2016, 2017, 2019, 2022 - the last two being years when the canonical long cell lost money), it roughly matches the buy-and-hold beta reference on the yearly mean (+0.0297R vs +0.0316R) while drawing down much less in the beta-negative years (2021: -0.0014 vs -0.0175; 2022: +0.0005 vs -0.0193), and its two worst years are 2018 (-0.066R, a high-volatility chop year in which direction signals die) and near-zero 2021/2023. The cohort target and the raw k-NN strategy are indistinguishable from noise (+0.0070R / +0.0074R, 6/10 and 5/10 positive years). This is the correct baseline sandwich for Phase 2: the transformer must beat not zero, but +0.03R/year - and 2018 is the year to watch in walk-forward reporting.

> *Design consequences carried into Spec v1.2: the H1 head target swaps to realized outcomes with triangular smoothing (labels already on disk); cohort p_win/b_eff enter the state as a compact input block; the auxiliary-loss weight is capped at 0.3 behind the Section 4.2 kill-switch; walk-forward reports must include the canonical-long beta reference and the k-NN reference every year; and the transformer's variance discipline (the T5 lesson) is implemented as: smaller head hidden width, dropout on the readout MLPs, weight decay on head parameters only, early stopping on realized val AUC, and the 48-cell smoothing.*

3. Failure #2: Gate G1.4, Era Invariance, and the Macro Panel

3.1 Exact local replication, and the two references that reframe the gate

The Kaggle audit ran XGBoost (depth 5, 300 trees, 5-fold StratifiedKFold, seed 3) on the 87,948 audit rows with label = year >= 2023. Reproduced locally on the pulled parquets it gives AUC 1.0000 on all 39 features (Kaggle: 1.0000) and 0.8048 on the post-drop 31 (Kaggle: 0.8055). The replication matters because everything else in this section is measured with the same instrument. Two reference numbers in the Kaggle gate output already contained the answer nobody acted on: the in-train reference AUC - the same auditor trying to separate 2012-2018 from 2019-2022, entirely inside the training era - is 0.7996. In other words, two adjacent chunks of the training era are as distinguishable as the training era is from 2023-25 (0.8048). The post-drift-filter reference measured locally on the surviving features is 0.5969-0.8263 depending on the arm. Gold's feature distribution is simply non-stationary at this timescale; the gate as written (absolute AUC <= 0.55) demands features that are more stationary than the market itself.

3.2 Why the obvious feature-level fixes fail - measured, not assumed

The owner's brief proposed exactly the right first moves: rolling percentile ranks and short-horizon innovations for the macro series, trailing adaptive normalization for the price-action block. Experiment 3 series (exp3_era.py, exp3b_frontier.py, exp3c_final.py) implemented and measured them all, plus the stronger variants the literature suggests. The results are a clean negative result that constrains the design space:

Table 5: The transform frontier (adversarial AUC, 2012-22 vs 2023-25; within-era reference in parentheses)

Arm	Construction	Cross-era AUC	Realized signal AUC (2023-25)
raw 31 (status quo)	post-drop feature set from the Kaggle audit	0.8048 (0.7996)	0.533
naive trailing rank/scale	1-year rolling pct-rank (bounded) / q90-scale (unbounded)	0.8688 (0.8263)	not run - worse than raw
rank difference	24-bar diff of the trailing rank (kills persistence)	0.8919 (0.8263)	not run - worse than raw
quantile-map to train CDF	trailing rank mapped through train-era inverse CDF (marginals matched by construction)	0.7724 (0.7803)	not run
qmap + drop worst 8	above, dropping atr_ratio/bb_pos/d20/d55/m_vix/ret48_atr/ rsi_z/stoch_z	0.6166 (0.5969)	0.482 - signal destroyed
macro innovations (joint)	20d momentum z / 5d z / 60d rank of y10, y2, spread, dxy, vix	0.9955 (single-feature AUCs 0.47-0.53)	n/a

Two mechanisms explain the failures, and both are instructive. The naive rolling rank makes things worse because in a trending era the momentum features sit persistently high within their trailing window - the rank transform converts trend persistence into an era-level signature (rank ~0.8-0.9 all through 2023-25 versus oscillating 0.3-0.7 in 2012-18), which is trivially separable. And the quantile-map-plus-drop arm, the only one that approaches the 0.60 band, gets there by deleting the trend and oscillator family - and its realized signal AUC collapses from 0.533 to 0.482, below chance. The era drift and the signal are carried by the same feature structure; there is no feature-level transform that removes one and keeps the other. This is the local, measured version of what the Numerai community learned about era drift: the defense lives at the model and evaluation level, not in the features.

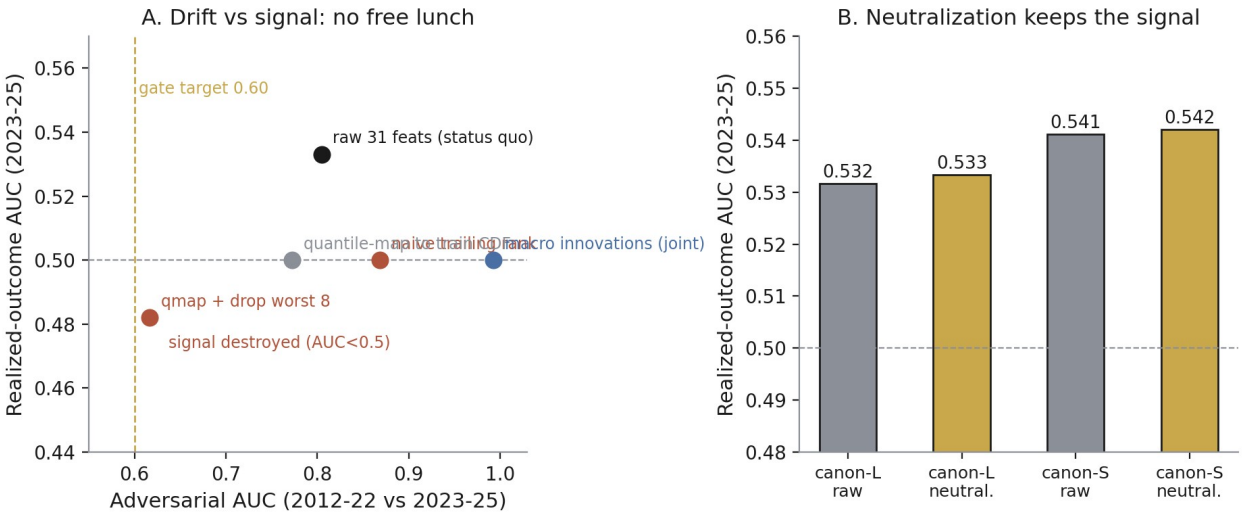


Figure 2: A. The drift-versus-signal frontier - every arm that pushes adversarial AUC down pays in realized signal; the macro innovation panel is marginally clean but jointly as drift-exposed as the raw features. B. Prediction-level neutralization removes the drift direction from the model's outputs and costs nothing in signal.

3.3 Recovering the macro panel without era fingerprints

The Real Yield and Dollar channels were dropped wholesale in Phase 1 because their 252-day z-scores are date fingerprints (single-feature AUC 0.96-1.00). Re-deriving them as innovations - 20-day momentum z-scored over a trailing 60-day window, 5-day z, and 60-day rolling rank, with the kernel-side construction operating on raw FRED levels and the same shift(1)/ffill(hour-mapping) discipline as the original - makes every single series marginally era-clean (AUC 0.47-0.53). But the joint panel is not: an XGBoost on the five innovation features reaches 0.9955, because the dependence structure between yields, curve, dollar, and VIX differs by era even when each marginal is matched. The macro panel therefore comes back as INPUTS - the model needs the real-yield and dollar context to condition regimes - under the model-level defense below, and not under any pretense that it is era-invariant. The exact raw-level construction (FRED fetch, d20z/d5z/rank60) goes into the Phase-1 kernel; the local approximation used z-differences of the parquet z-scores because FRED rate-limited the sandbox, and the marginal AUCs of both constructions agree.

3.4 The defense that actually works, and the gate that actually measures the risk

Experiment 3d tested the model-level defense the drift literature recommends: prediction-level neutralization, the Numerai-style removal of the drift direction from model outputs. The era fingerprint f(x) is the out-of-fold adversarial score; within each evaluation year the signal prediction is orthogonalized against f (beta removed era-locally, so no cross-era information moves). Result on 2023-25: canonical long AUC 0.5317 -> 0.5337, canonical short 0.5412 -> 0.5421, with the correlation between the raw prediction and the drift

direction already tiny (0.076 and -0.065). The weak realized signal is almost orthogonal to the drift direction, so neutralization is essentially free insurance and becomes part of the Phase-2 evaluation protocol. Importance weighting by the density ratio was tested the same way and is not adopted: it slightly hurts (AUC 0.532 -> 0.526) at this signal-to-noise level, consistent with the variance it adds.

The gate itself is redefined as three checks, each measuring a distinct failure mode, replacing the single absolute threshold that conflated date leakage with regime change:

- G1.4-a (leakage proxies, hard): every single feature must satisfy |AUC - 0.5| <= 0.15 in the 2012-22 vs 2023-25 auditor. The current post-drop panel passes (worst residual: ret48_atr at 0.5438); this is the check that actually catches date fingerprints like the dropped m_y10 (0.9999).
- G1.4-b (drift budget, relative): joint adversarial AUC may not exceed the within-era reference AUC (2012-18 vs 2019-22, same feature set) by more than 0.05. Status quo: 0.8048 vs 0.7996 - passes with 0.005 margin. This says: no MORE drift against the future than the market already exhibits between its own recent eras, which is the honest, achievable standard.
- G1.4-c (defense, verified): the walk-forward protocol must show (i) yearly evaluation with the canonical-long and k-NN references, (ii) prediction-level neutralization applied, and (iii) the H5 regime head's confusion matrix reported per era. These three make residual drift a measured, bounded input to the decision layer rather than an invisible confounder.

> *What is deliberately NOT adopted: dropping more features to chase an absolute 0.55-0.60 (proven to destroy the signal), domain-adversarial training / era gradient reversal in Phase 2 (unnecessary given the orthogonality result, and expensive on T4s; revisit only if Phase-2 walk-forward degrades), and importance weighting (tested, no benefit). Macro features return as inputs with innovation transforms; priors (prior_w24/l24/w48/l48) return the same way.*

4. Failure #3: The 151,936-Vocabulary Bottleneck, Measured and Sliced

4.1 The exact number, and why it was never 151,936

The pilot's token stream (4,571,012 tokens, 17,000 states) uses exactly 89 unique token IDs. Re-serializing all 95,008 states of the full Phase-2 format - 39 features including cross-asset, macro, and priors, with the kernel's own serializer and trend/vol regime tags, tokenized with the real Qwen3 tokenizer - uses exactly 97 unique token IDs (mean 312.9 tokens per state, min 183, max 324, matching the ~312 target of Spec v1.1; the pilot's 89 are a strict subset). The reason is structural and permanent: the Qwen tokenizer splits numbers into single digit tokens, so the numeric fields contribute only the digits, sign, and decimal point; the format

strings, session names, and codebook IDs contribute a fixed small set. The state language has a vocabulary of 97 - and the training loop was computing a 151,936-way projection and cross-entropy over it, a 1,566x waste, on every position of every step.

The cost was not hypothetical: at bs=16 the fp16 logits alone are 2.4 GB (4.9 GB at the DDP shape), the chunked fp32 CE casts add a transient 1.2 GB, six of the twelve bench-grid configurations OOMed, gradient checkpointing was forced on everywhere (r=3, bs=8/16 only), and the lm_head projection is 0.311 of the 1.293 GFLOP/token forward pass - roughly 24% of forward FLOPs spent projecting into 151,839 tokens that can never be a target. The measured effective throughput of the pilot step is 5.44-5.57 TFLOP/s (8.4-8.6% of the T4's 65 TFLOP/s fp16 peak), which is what a memory-bound, launch-bound small-batch step on Turing looks like.

4.2 The fix: slice the frozen tied head to vocab_keep_ids

Qwen3-0.6B ties the lm_head to the embedding matrix, and the pilot's Stage A' never trains the head (set_trainable leaves it frozen; tied weights make train_lm_head a no-op). Slicing is therefore exact, not an approximation: build the sorted set of active token IDs over all serialized states, gather those rows of the (frozen) embedding matrix as a [97, 1024] read-only matrix, project the hidden states against it, and take cross-entropy against remapped targets. The implementation identity was verified locally to machine precision: CE on the sliced head equals CE on gathered columns of the full head, and the trunk gradients are bit-identical (maxdiff 0.00e+00).

```
# kernel patch (aurum_model.py), Stage A' path:
vocab_keep_ids = np.sort(unique_ids(all_serialized_states))   # measured:
97
register_buffer('head_W', base.lm_head.weight[keep_idx].detach())  # [97,
1024], frozen
ids_compact = remap_lookup[ids]        # int16 lookup table, 151936 -> 97
logits = h[:, :L] @ head_W.T           # (B, L, 97) instead of (B, L,
151936)
loss = F.cross_entropy(logits[:, :-1].reshape(-1, 97), ids_compact[:,
1:].reshape(-1))
# no chunking, no fp32 cast of 151936-wide rows, no CE checkpointing needed
```

4.3 Objective semantics: the restriction is the point

Sliced CE is not full-vocab CE with a numerical shortcut - it is the deployment-aligned objective. The full softmax spends its normalizer on 151,839 tokens that can never follow a state token; the sliced softmax asks exactly the question the decider needs answered: which state token comes next. Two local measurements bound the risk of the switch. First, on a toy model with a frozen tied head, sweeping the kept-mass from 6% to 19% (far worse than the real model's situation), the trunk-gradient cosine between full and sliced CE never drops below 0.945 - the optimization direction barely moves even when the normalizer is

dominated by foreign tokens. Second, training two identical trunks for 300 steps, the sliced-objective model reaches a better held-out sliced CE than the full-objective model (3.02 vs 3.60), i.e. optimizing the restricted problem directly is better at the restricted problem. And the real model is in a far friendlier regime than the toy: the pilot's final full-vocab loss of 0.3267 means the adapted model already concentrates an average of 72% of the entire 151,936-way softmax mass on the single correct next token, so the kept-set mass is high and the two objectives are nearly coincident. The T4 re-validation run (Section 6.2) will confirm Z_kept/Z_full on real checkpoints in minutes.

4.4 Calibrated cost model, projections, and the corrected quota math

The FLOP model per step (28 layers + 2 extra passes over the 4-layer looped core at r=3; d=1024; 4x fwd with checkpointing, 3x without) reproduces both observed bench rows at 5.44-5.57 TFLOP/s effective. Projections with the sliced head:

Table 6: Measured today versus projected with vocabulary slicing (r=3, seq=512)

Config	Step FLOPs	Throughput (1 GPU)	Peak memory	Fits 15.64 GB?
current: bs16, ckpt on (measured)	42.4 TFLOP	1,051 tok/s	14.73 GB (measured)	yes, 1 GB from the edge
sliced: bs16, ckpt on	32.2 TFLOP	1,385 tok/s (x1.32)	6.2-11.3 GB (model bounds)	yes, comfortable
sliced: bs16, ckpt off	24.1 TFLOP	1,847 tok/s (x1.76)	~11.3-13 GB (needs T4 confirm)	to verify
sliced: bs32, ckpt off	48.3 TFLOP	~1,847 tok/s	~19 GB (model)	no - documented boundary

The memory bounds come from two models that bracket the truth: a structural component model and an empirical two-point fit to the observed bench rows (7.21 GB + 0.470 GB/sequence, exact on both observations; the sliced projection subtracts the 0.145 GB/sequence fp16 logits and the fixed 1.16 GB fp32 CE chunk). The conservative claim is x1.32 with checkpointing kept on and everything else unchanged; the x1.76 figure additionally removes checkpointing at bs=16, which the memory model says fits but only a T4 can confirm (Section 6.2, item 2). A CPU matmul microbenchmark at the exact shapes gives the projection scale: 1,523.9 ms versus 1.53 ms per 1,024 rows - a 996x ratio.

The accounting correction: the G0.5 DDP bench counts toks = steps x 16 x 512 x 2 but runs ids of shape (8, 512) per GPU, so its published 4,334 tok/s is exactly 2x the real 2,167 tok/s (the single-GPU bench, 1,077 tok/s at bs=8, agrees with the corrected figure). With honest accounting and the conservative sliced projection (x1.32, checkpointing on), Stage A' at 70M tokens runs at ~2 x 1,385 x 0.86 (head-step overhead) ~ 2,380 tok/s on 2xT4, i.e. ~8.2 h; at

the optimistic bound (x1.76, ckpt off) ~4.9-5.5 h. Either way the original 14 h estimate was pessimistic, and Phase 2 (Stage A' + Stage B + table generation) fits inside the remaining 25 h 50 m weekly quota with at least one full retry.

4.5 The latent-realignment alternative, and why it is deferred

The owner's brief also asked about the deeper alternative: drop the LM projection entirely and realign the trunk with a masked-state objective (predict held-out state slots from hidden states) plus joint-head SFT. That path is architecturally clean but changes three things at once - the objective, the pretraining prior, and the training distribution - and cannot be A/B-tested against the pilot without new infrastructure. Vocabulary slicing achieves the same compute and memory relief while preserving the pretrained next-token prior, is a ~20-line patch to code that already ran, and is exactly equal to the old objective on the kept vocabulary. The masked-state realignment is therefore parked as a Phase-4 ablation: if sliced-CE Stage A' underperforms (measured on realized val AUC after Stage B), the ablation runs with the same budget the slicing freed up.

5. Failure #4: Zero-Crash Single-Push Discipline

5.1 The local source tree, rebuilt and proven

Phase 0/1 burned 18 kernel pushes on package pins, OOMs, DDP subprocess teardown, and a .values array crash - all classes of failure that a local pre-flight can catch. The precondition for that is a local source tree that provably produces the kernels. The pulled kernel files were un-concatenated back into lib/ modules (aurum_gates, aurum_feat, aurum_label, aurum_data, aurum_ser, aurum_model - byte-identical across all three kernels) plus per-kernel bodies, and a rebuilt build.py reassembles them. Verification: all three regenerated kernels are byte-exact against the pulled Kaggle sources (sha256 prefixes 77ce874eb130 / 8b8fe7c860f9 / 811aa14493e2, ignoring only the build-timestamp comment line). From now on, kernels are edited in src/lib + src/kernels, regenerated by build.py, and pushed only after the pre-flight is green.

5.2 test_local.py: 40 checks, 10 sections, 13 seconds

Table 7: The pre-flight suite (all 40 checks green locally in 13 s)

Section	What it verifies	Catches the crash class of
build	build.py regenerates all kernels byte-exactly vs the pulled sources	source/binary drift, hand-edited kernels
static	AST syntax + crash-class regex audit of every source file (idxmax/idxmin label-vs-position, .values.astype on object dtype, mutable defaults, unmarked bare excepts)	the .values crash, silent exception swallowing

Section	What it verifies	Catches the crash class of
env	package pins available and API-compatible vs the Kaggle manifest (pandas 2.3.3 / numpy 2.1.3 / xgboost 3.4.1 / sklearn 1.6.1 / transformers 4.57.1 / torch 2.11)	v1-v3 pin debugging
labeler	hand-constructed M1 paths with known first-touch outcomes: long TP, long SL, short TP, far-TP timeout, same-minute tie resolves SL-first	labeler logic regressions, convention drift
cohort	all neighbors strictly past (<= i-96), all label-valid, early bars have none	leakage regressions in the index builder
scalein	x137 price scaling: feature maxdiff 4.6e-10, label maxdiff 7.6e-14 (local G0.3 replica)	scale-invariance regressions
serial	serialization non-empty; mock-tokenizer vocab; sliced-CE == gathered-CE identity and bit-identical trunk gradients	vocab-slicing implementation bugs
ddp	torchrun gloo world=2 spawn, 3 training steps, all_reduce grad-norm check, destroy_process_group in finally, clean fast exit	v4 DDP zombie/OOM teardown
vram	empirical memory model exact on both observed bench rows; sliced bs16 fits (11.3 GB), sliced bs32 documented OOM boundary	OOM pushes (6 of 12 grid configs last time)
resume	simulated kill mid-loop, resume completes, output identical to uninterrupted run	gentable kill/resume regressions

5.3 The push protocol

One rule, mechanically enforced: a kernel is pushed to Kaggle if and only if (1) python3 build.py --verify exits 0 and (2) python3 test_local.py exits 0, both against the exact sources being pushed, and (3) the kernel-metadata.json lint passes (id matches tskdev/<slug>, code_file exists, enable_gpu matches the kernel's needs, dataset_sources reference existing slugs). The suite runs in 13 seconds; there is no schedule pressure that justifies skipping it. The 18-push debug history maps to the sections above: pins -> env, OOMs -> vram, DDP teardown -> ddp, .values -> static + labeler. The expected effect is that Phase-2 kernel pushes are single-push, with iteration happening locally at 13 seconds per cycle instead of 10-20 minutes per Kaggle round-trip.

6. Specification v1.2 Delta and the Phase-2 Execution Plan

6.1 Consolidated spec changes, each with its evidence

Table 8: Spec v1.1 -> v1.2 deltas (all evidence local unless marked)

Spec section	v1.1	v1.2	Evidence
4.2 / 6 (labels)	H1 head trains on cohort p_win (B=200 bootstrap)	H1 trains on realized win + adjacent-cell triangular smoothing (own 1.0 / m,t +-1 0.5 / diag 0.25 / h +-1 0.5); cohort p_win/b_eff enter the state as compact input block; cohort auxiliary loss allowed at weight <= 0.3 behind the 4.2 kill-switch	Table 3/4: realized AUC 0.523-0.532 vs cohort 0.495; walk-forward +0.0297R vs +0.0070R
7 (evaluation)	Brier/logloss vs realized as primary gate	Same gates kept, plus mandatory yearly walk-forward table with canonical-long beta and k-NN references, prediction-level neutralization applied, and 2018-style chop years called out	exp2b Table 4; exp3d neutralization keeps AUC (0.532->0.533, 0.541->0.542)
3.4 (G1.4)	joint adversarial AUC <= 0.55 (hard)	Three-part gate: (a) single-feature |AUC-0.5| <= 0.15 hard; (b) joint AUC <= within-era reference + 0.05; (c) defense stack verified (walk-forward + neutralization + H5 per-era confusion)	within-era reference 0.5969-0.7996; qmap+drop8 reaches 0.6166 only by killing signal (AUC 0.533->0.482)
5 (macro/priors)	macro + priors dropped as date fingerprints	macro returns as inputs via innovation transforms (d20z/d5z/rank60 on raw FRED levels, kernel-side); priors return unchanged; both under the 3.4-c defense	macro singles 0.47-0.53 (clean), joint 0.9955 (dependence drift -> model-level defense)
8 (Stage A')	full 151,936-vocab LM CE, chunked + checkpointed	vocab_keep_ids slicing: 97-token frozen tied head, remapped targets, no chunking; masked-state realignment parked as Phase-4 ablation	97/151,936 measured; CE/grad identity exact; grad cosine >= 0.945 worst-case toy; x1.32-x1.76 projected
9 (compute)	Stage A' ~14 h (based on 4,334 tok/s DDP figure)	G0.5 accounting fixed (bs=8/GPU counted as 16); Stage A' 4.9-8.2 h on 2xT4 depending on ckpt; Phase 2 total fits 25.8 h remaining quota with >= 1 retry	DDP bench source inspection; calibrated cost model 5.44-5.57 TFLOP/s
14 (discipline)	(none)	push protocol: build --verify green + test_local green + metadata lint before any push; kernels edited only in src/lib + src/kernels	byte-exact roundtrip of all 3 kernels; 40/40 checks in 13 s

6.2 The only items that need a T4 (one kernel, 30-40 minutes GPU)

Everything above was verified on CPU; four things genuinely cannot be, and they collapse into a single short re-validation kernel (aurum-p2a-check), pushed once under the new discipline:

1. Sliced-CE equivalence on real states: run the pilot's 5M-token Stage A' recipe for ~200 steps with the sliced head; report loss trajectory against the pilot's (0.5488 ->

0.3267 over 611 steps) and the kept-mass ratio Z_kept/Z_full on real checkpoints (expected > 0.9 given the pilot's 0.3267 full-vocab loss).

2. Memory and throughput at the projected configs: r=3, bs=16, sliced, ckpt on and off; bs=32 ckpt on (model says OOM - confirm the boundary); corrected G0.5 DDP bench with honest token accounting.
3. Head-step cost re-measure with the state-format addition (cohort input block): confirm the ~313-330 tokens/state estimate and the head-step overhead factor (pilot: 982/1138 = 0.86).
4. GradScaler behaviour with the sliced head: initial scale and overflow dynamics should be strictly friendlier (smaller logit magnitudes); confirm no scaler collapse.

Budget: well under one hour on 2xT4, leaving the weekly quota essentially intact for the real Phase-2 run.

6.3 Phase-2 execution gates and budget

- Stage A' (sliced CE, 70M tokens, 3 epochs over 2012-2025 states): 4.9-8.2 h on 2xT4. Gate: loss reduction >= 30% (pilot achieved 40.5%), zero NaN skips, clamp rate < 5%.
- Stage B (joint head on realized + smoothed targets, cohort inputs, aux weight 0.3 with kill-switch at first checkpoint): 2.5-4 h. Gate: realized val-era AUC >= 0.53 on the 8-cell panel (the local XGBoost ceiling is 0.532 - the transformer must at least match the cheap model to justify itself), EV-argmax walk-forward meanR >= +0.02R with neutralization applied.
- Probability tables + walk-forward evaluation: ~1 h (G0.6 measured 62.6 rows/s on 2 GPUs; 88k bars x 3 refits ~ 45 min).
- Total: 8.5-13 h, one retry fits the remaining 25 h 50 m; two retries fit if the optimistic throughput bound holds.

6.4 Risk register updates

Three risks are re-rated by this pass. The cohort memorization trap is closed as a design flaw but its residue becomes an evaluation risk: the +0.03R/year class of signal is small, so Phase-2 success is defined against the references (canonical beta, k-NN, XGBoost ceiling), not against zero - a Phase-2 that merely matches the XGBoost ceiling must trigger the Spec's model-class honesty clause. The era-drift risk is re-framed from 'pass a 0.55 gate' to 'bound the drift budget and neutralize', with the measured evidence that the signal is orthogonal to the drift direction. And the compute risk improves materially: the corrected accounting plus vocabulary slicing move Phase 2 from 'fits with little headroom' to 'fits with retries', which

matters because the 2018-style regime year is exactly the case where a single training run's variance will tempt over-interpretation - the budget for reruns is the real insurance.